Práctica de bases de datos NewSQL y NoSQL: CockroachDB y MongoDB

Cristian Alejandro Perdomo Rivas

Universidad de Cundinamarca

25 de octubre de 2026

Repositorio del proyecto:
`https://github.com/CAlejandroPerdomo/UDEC-databases-sql-nosql`

Resumen

Este informe documenta la práctica de laboratorio sobre bases de datos NewSQL y NoSQL. En la primera parte se trabajó con CockroachDB, un motor NewSQL distribuido y compatible con SQL, accedido desde el cliente DBeaver contra un clúster alojado en CockroachDB Cloud. En la segunda parte se trabajó con MongoDB, una base de datos NoSQL orientada a documentos, mediante MongoDB Compass y el shell `mongosh` contra un clúster de MongoDB Atlas. Sobre una base de datos académica *Universidad* se ejecutaron las operaciones básicas de creación de esquema y tabla (o colección), inserción, consulta, actualización y eliminación de registros, verificando en cada caso el resultado mediante las capturas de pantalla de las herramientas. La práctica permitió contrastar el modelo relacional estricto de CockroachDB con el modelo de documentos flexible de MongoDB, evidenciando cómo ambos motores cubren el mismo ciclo CRUD con enfoques y lenguajes distintos.

Palabras clave: bases de datos, NewSQL, NoSQL, CockroachDB, MongoDB, CRUD, DBeaver, MongoDB Compass.

1 Introducción

Las bases de datos relacionales dominaron durante décadas el almacenamiento estructurado de información, pero la necesidad de escalar horizontalmente y de manejar volúmenes y variedades crecientes de datos impulsó el surgimiento de nuevas familias de sistemas de gestión de bases de datos. Por un lado, los sistemas NoSQL renuncian al esquema rígido y al modelo relacional para ofrecer mayor flexibilidad y distribución; por otro, los sistemas NewSQL buscan conservar la garantía de transacciones ACID y el lenguaje SQL de las bases relacionales, a la vez que incorporan la escalabilidad horizontal propia de los sistemas distribuidos modernos (Tankoano, 2025).

CockroachDB es un representante característico de la categoría NewSQL: un motor SQL distribuido, tolerante a fallos y georreplicado, que expone una interfaz compatible con PostgreSQL y mantiene transacciones serializables sobre múltiples nodos (Reid, 2022). MongoDB, en cambio, es la base NoSQL orientada a documentos más difundida: almacena la información en documentos BSON con esquema flexible y la organiza en colecciones, lo que la hace adecuada para datos heterogéneos y en evolución (Carvalho, Sá, & Bernardino, 2022).

El objetivo de esta práctica es aplicar, sobre un mismo caso de estudio, el ciclo básico de operaciones de datos (crear, insertar, consultar, actualizar y eliminar) en los dos paradigmas, y contrastar sus diferencias en cuanto a modelo, lenguaje y herramientas. El documento se apoya en los scripts y en las capturas de pantalla generadas durante la sesión, que constituyen la evidencia de los procedimientos realizados.

2 Objetivos

2.1 Objetivo general

Aplicar el ciclo básico de gestión de datos (creación, inserción, consulta, actualización y eliminación) sobre una base de datos académica, empleando un motor NewSQL (CockroachDB) y un motor NoSQL (MongoDB), y documentar las diferencias entre ambos paradigmas.

2.2 Objetivos específicos

- Establecer la conexión con los clústeres en la nube de CockroachDB Cloud y MongoDB Atlas desde las herramientas de cliente correspondientes.
- Crear la estructura de datos (esquema y tabla, o base y colección) en cada motor.
- Insertar, consultar, actualizar y eliminar registros en ambos sistemas y verificar los resultados.
- Comparar el modelo relacional de CockroachDB con el modelo de documentos de MongoDB a partir de la evidencia recolectada.

3 Marco teórico

3.1 Bases de datos NewSQL y CockroachDB

El término NewSQL agrupa a los motores relacionales que combinan el soporte del lenguaje SQL y las transacciones ACID con una arquitectura distribuida y escalable horizontalmente, superando las limitaciones de escalado de los gestores relacionales tradicionales (Tankoano, 2025). CockroachDB implementa este enfoque mediante una arquitectura de nodos replicados que soportan transacciones serializables distribuidas, tolerancia a fallos y replicación geográfica, manteniendo compatibilidad con el protocolo de PostgreSQL (Reid, 2022; Cockroach Labs, 2025).

3.2 Bases de datos NoSQL y MongoDB

Las bases NoSQL comprenden varios modelos (clave-valor, documental, columnares y de grafos) que sacrifican parte de las garantías relacionales en favor de la escalabilidad y la flexibilidad de esquema (Carvalho et al., 2022). MongoDB es un sistema orientado a documentos que organiza la información en colecciones de documentos BSON; cada documento puede tener una estructura propia, lo que facilita modelar datos heterogéneos y realizar cambios de esquema sin migraciones costosas (MongoDB, Inc., 2025).

4 Desarrollo

El desarrollo se presenta paso a paso, en el orden en que se ejecutó cada operación, y cada captura de pantalla se acompaña de su explicación. La Parte A corresponde a la práctica NewSQL con CockroachDB y la Parte B a la práctica NoSQL con MongoDB.

4.1 Parte A: NewSQL con CockroachDB

La práctica NewSQL se ejecutó desde DBeaver contra el clúster de CockroachDB Cloud. El script de referencia es `NewSQL/Script/script.sql`.

Paso 1: conexión al clúster. Se configuró en DBeaver la conexión al clúster `oracle-test-35695.j77` (puerto 26257) mediante el controlador JDBC de CockroachDB, apuntando a la base de datos `Universidad` (Figura 1).

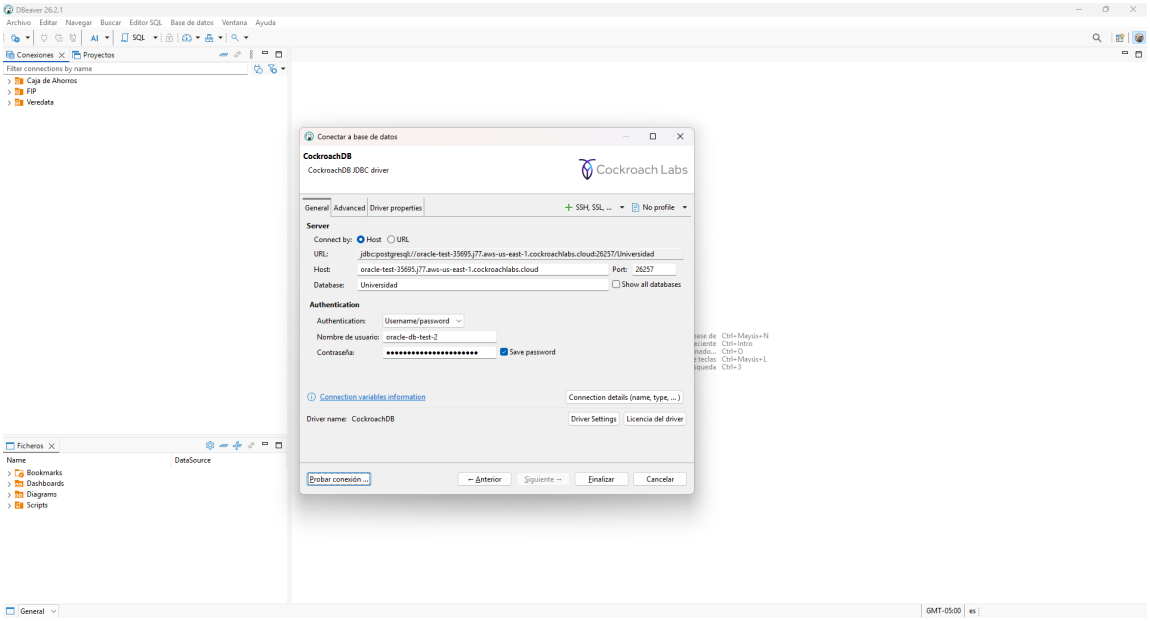


Figura 1. *Configuración de la conexión a CockroachDB Cloud desde DBeaver.*

Paso 2: creación de la base de datos. Con la conexión establecida se ejecutó `CREATE DATABASE Universidad;`, cuyo panel de estadísticas reportó 0 filas afectadas y un tiempo de ejecución de 3,071 s (Figura 2).

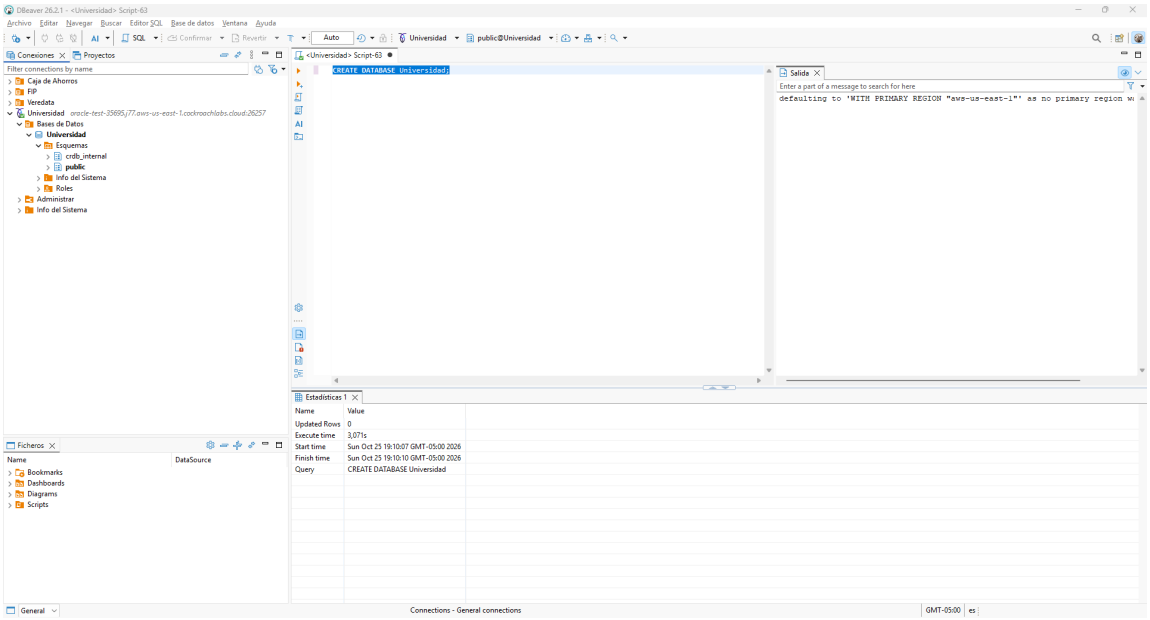


Figura 2. *Creación de la base de datos* ***`Universidad`****.*

Paso 3: creación del esquema. Se creó el esquema `academico` con `CREATE SCHEMA academico;`, que queda visible en el árbol de navegación del cliente (Figura 3).

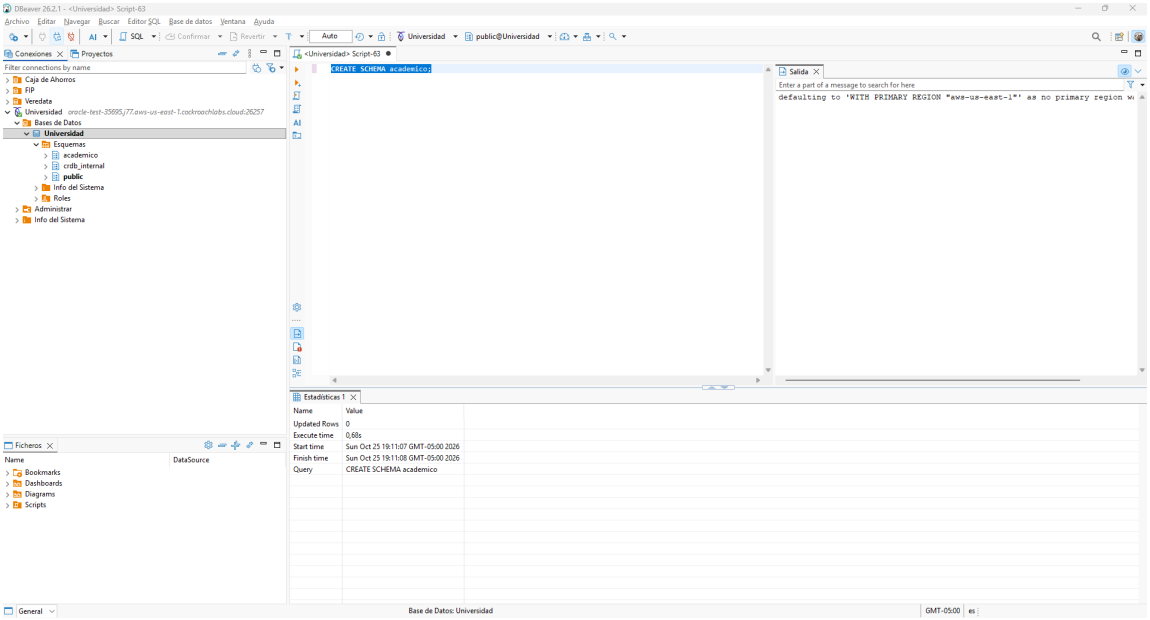


Figura 3. *Creación del esquema* `academico`.

Paso 4: creación de la tabla. Se creó la tabla `academico.estudiantes` con las columnas `id`, `nombre`, `edad`, `carrera` y `semestre` (Figura 4).

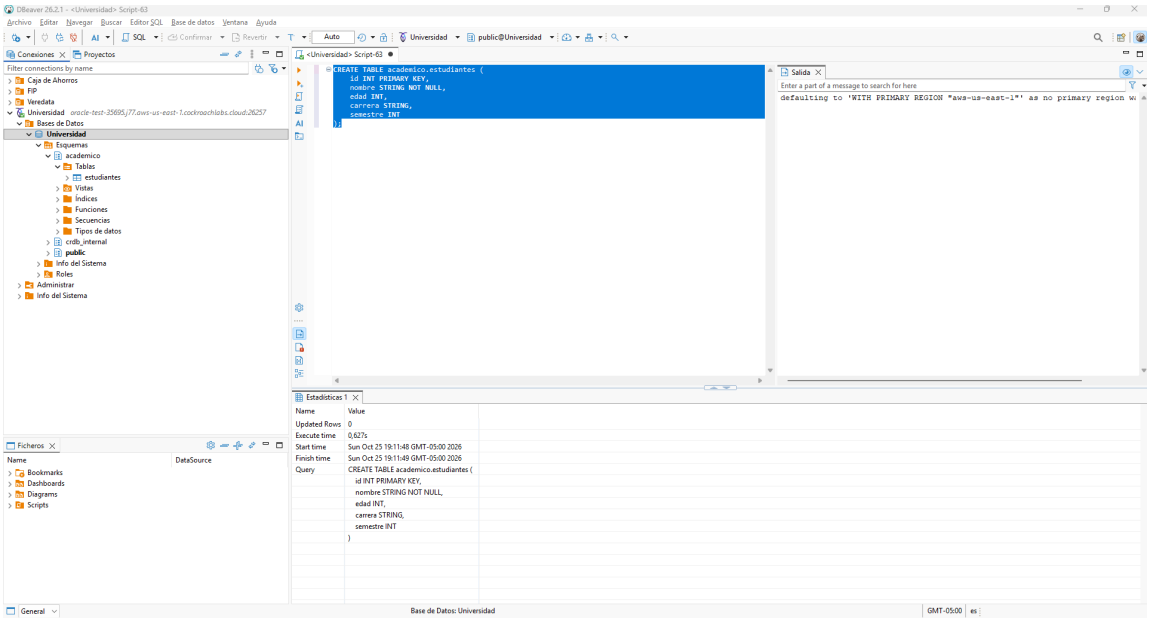


Figura 4. *Creación de la tabla* `academico.estudiantes`.

Paso 5: inserción de registros. Se insertaron tres registros con `INSERT INTO`; el panel confirma 3 filas afectadas y la rejilla de datos muestra los tres estudiantes (Ana, Carlos y Laura) (Figura 5).

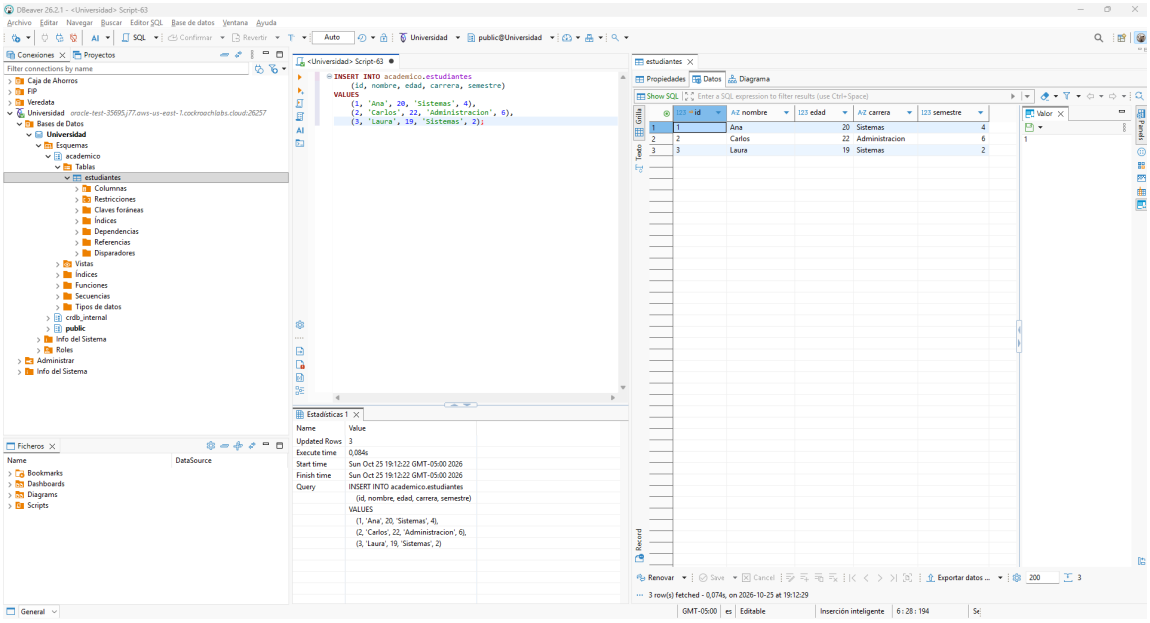


Figura 5. *Inserción de tres registros en* `academico.estudiantes`.

Paso 6: consulta de todos los registros. La sentencia `SELECT * FROM academico.estudiantes;` devolvió los tres registros (3 filas obtenidas) (Figura 6).

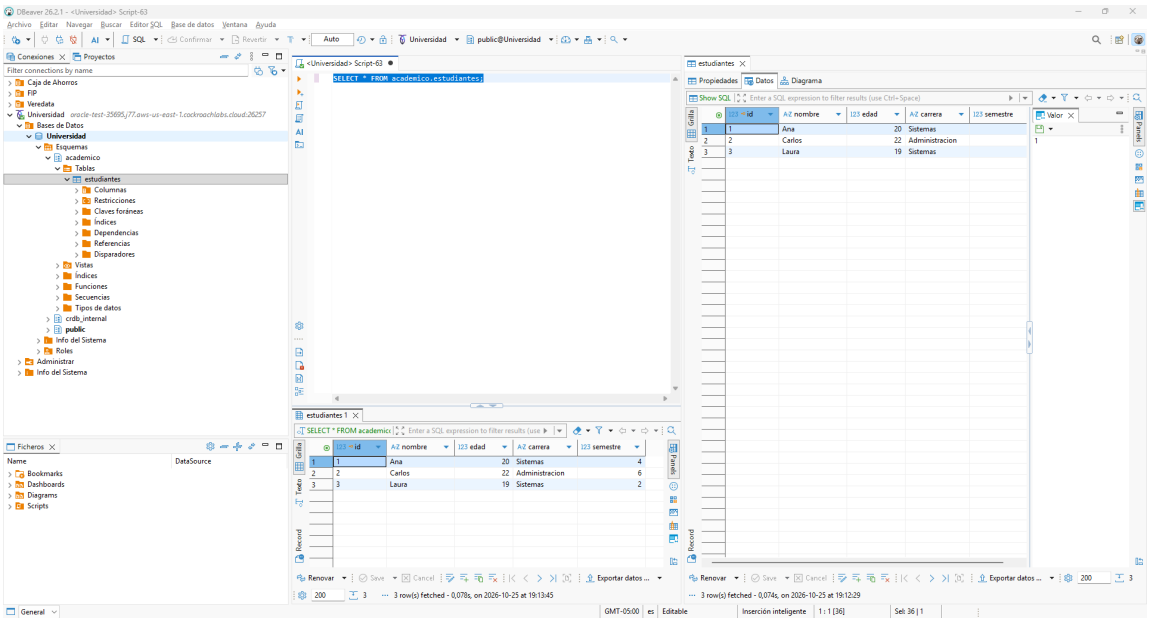


Figura 6. *Consulta* `SELECT * FROM academico.estudiantes`.

Paso 7: consulta filtrada. La consulta `SELECT * ... WHERE carrera = 'Sistemas';` devolvió dos filas (Ana y Laura) (Figura 7).

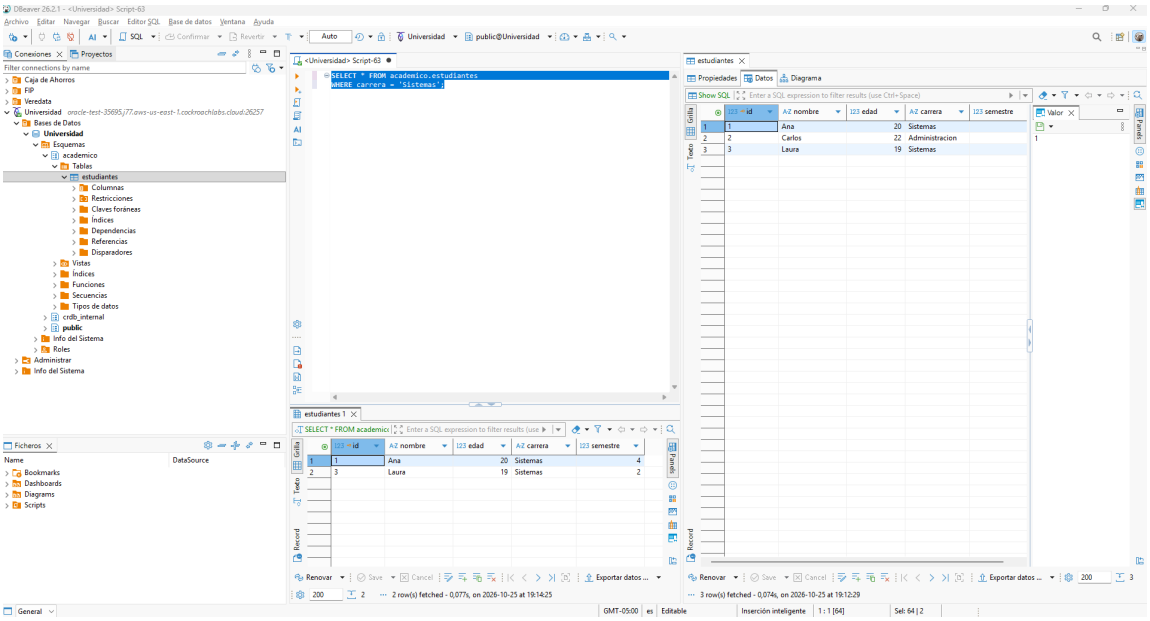


Figura 7. *Consulta filtrada por* `carrera = 'Sistemas'`.

Paso 8: actualización de un registro. La sentencia `UPDATE ... SET edad = 21, semestre = 5 WHERE id = 1;` actualizó una fila y la rejilla muestra a Ana con la edad y el semestre modificados (Figura 8).

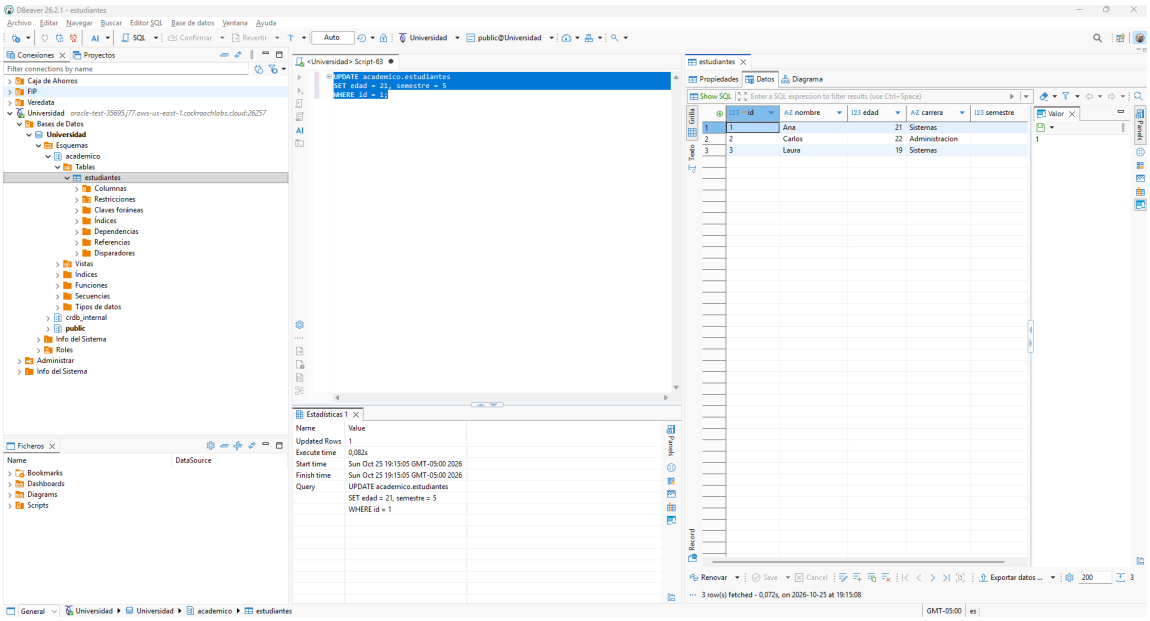


Figura 8. *Actualización del registro con* `id = 1`.

Paso 9: eliminación de un registro. La sentencia `DELETE FROM ... WHERE id = 3;` eliminó una fila y la consulta posterior devuelve dos filas (Ana y Carlos) (Figura 9).

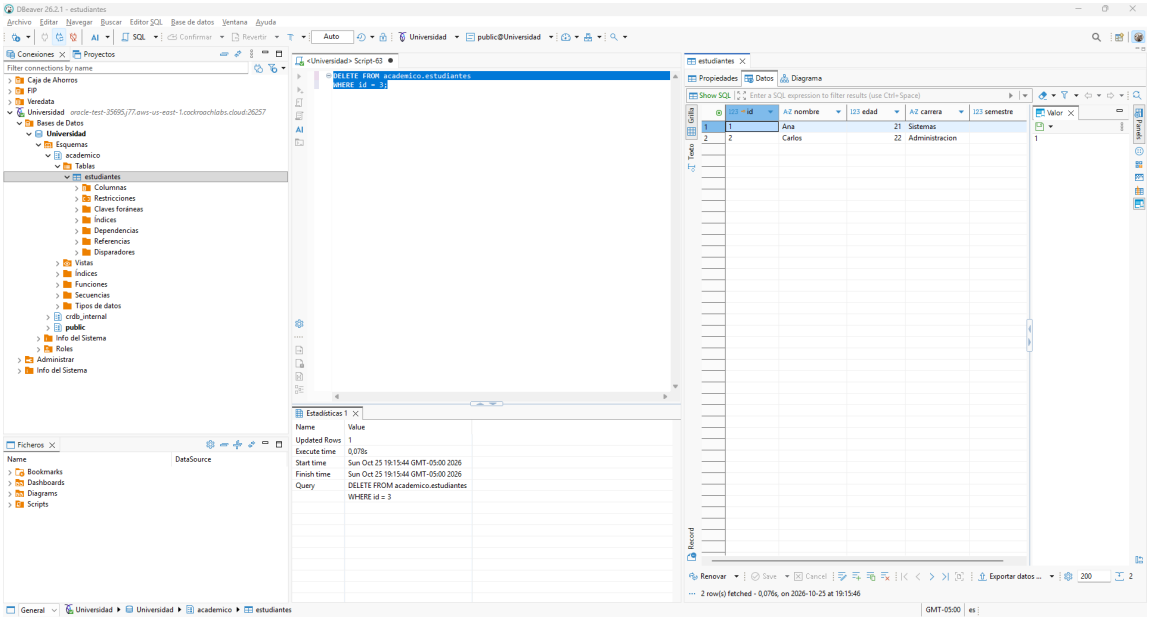


Figura 9. *Eliminación del registro con* `id = 3`.

Paso 10: estado del clúster. El panel de control de CockroachDB Cloud muestra el clúster `oracle-test` utilizado (AWS us-east-1, 2 vCPU, 2,09 MiB usados) con sus métricas de rendimiento y latencia (Figura 10).

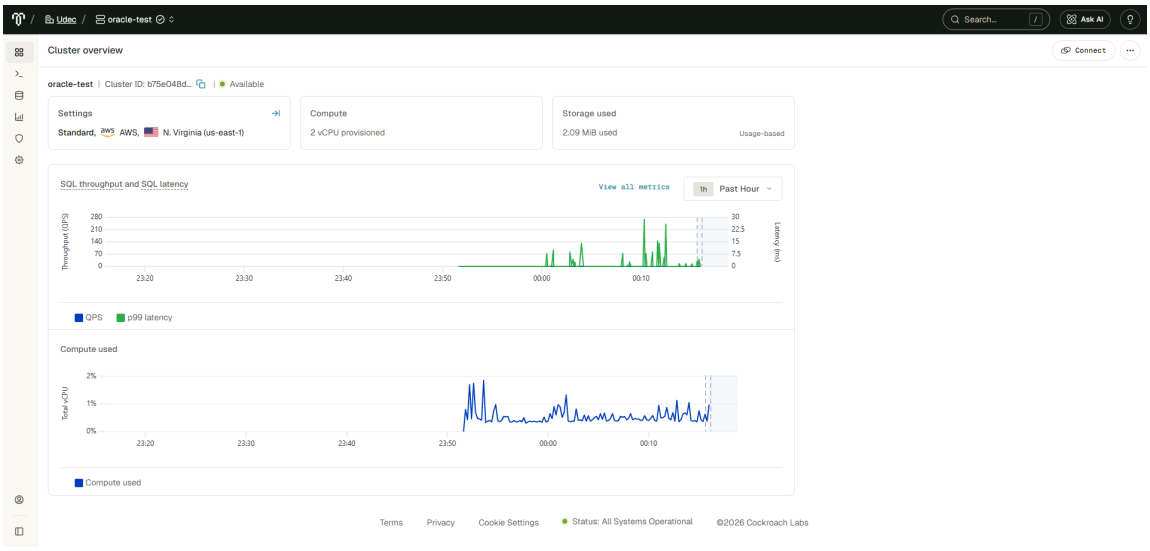


Figura 10. *Panel de control del clúster en CockroachDB Cloud.*

4.2 Parte B: NoSQL con MongoDB

La práctica NoSQL se ejecutó mediante MongoDB Compass y el shell `mongosh` contra el clúster de MongoDB Atlas. El script de referencia es `NoSQL/Script/script.js`.

Paso 1: inicio de Compass. Se abrió MongoDB Compass, cuya pantalla de bienvenida no muestra aún ninguna conexión configurada (Figura 11).

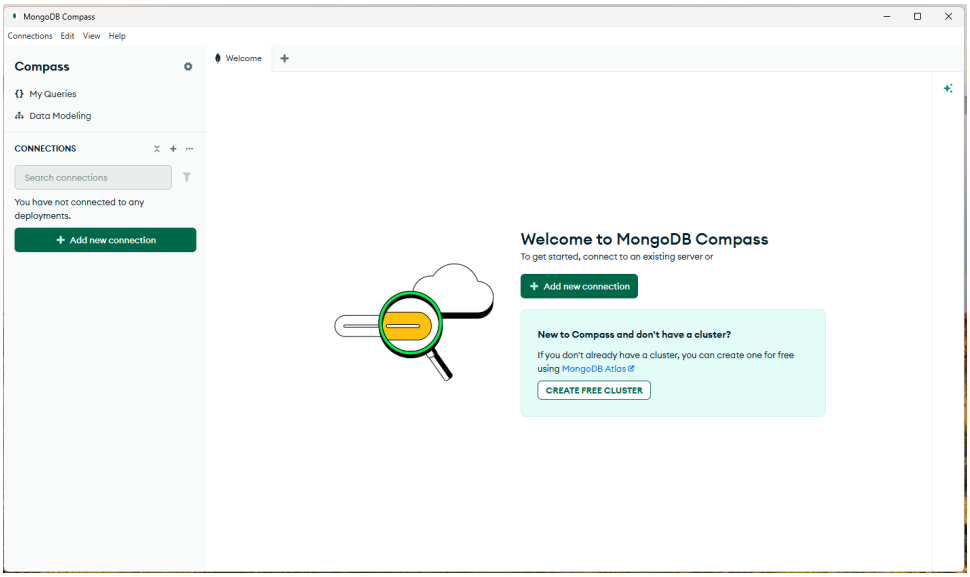


Figura 11. *Pantalla de inicio de MongoDB Compass.*

Paso 2: configuración de la conexión. Se creó una nueva conexión (`Mongo Server`) mediante la cadena `mongodb+srv://...@mongotest.ulbo4mr.mongodb.net/` del clúster de Atlas (Figura 12).

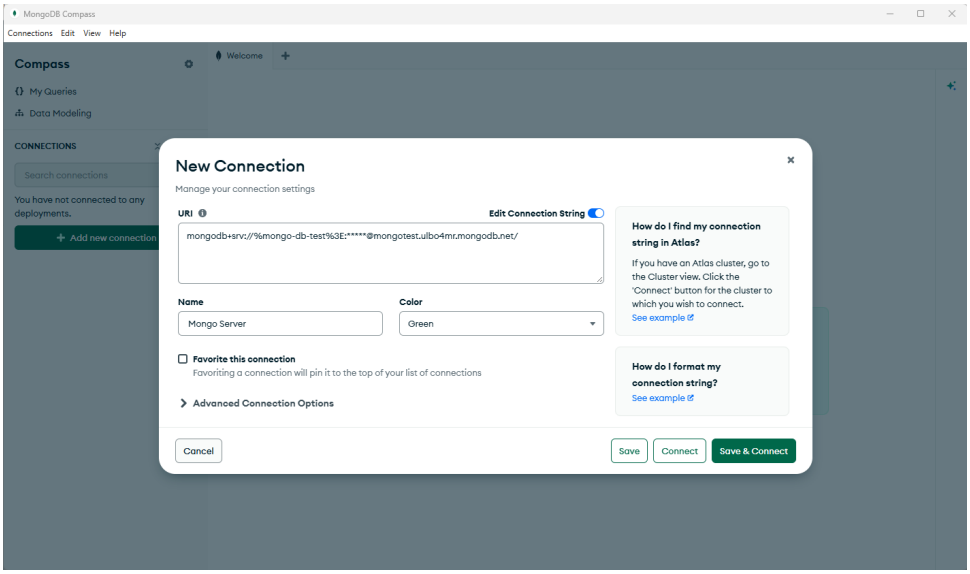


Figura 12. *Configuración de una nueva conexión en Compass.*

Paso 3: verificación de la conexión. En `mongosh` se ejecutó `db.runCommand({ ping: 1 })`, que devolvió `{ ok: 1 }` y confirmó la conexión con el clúster (Figura 13).

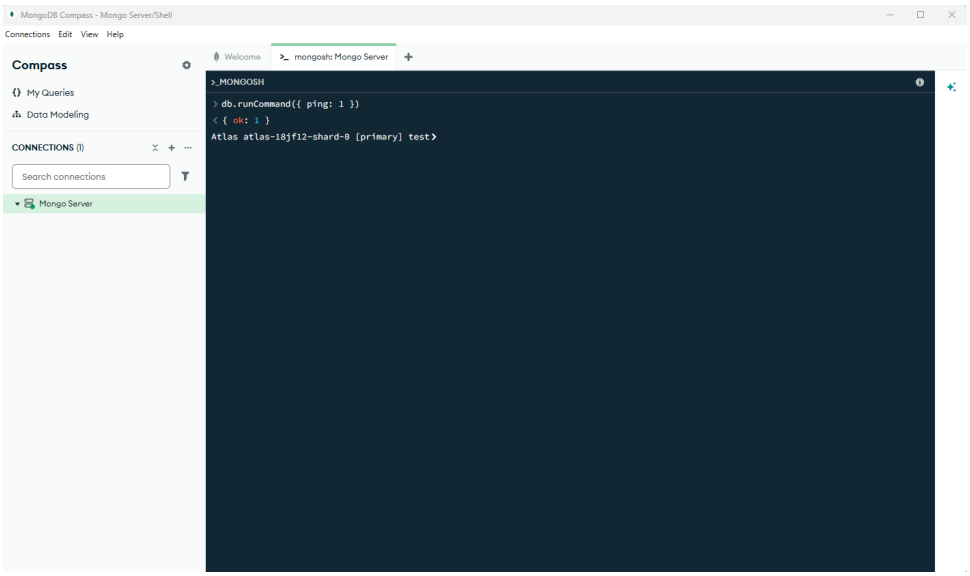


Figura 13. *Verificación de la conexión con* ***`ping`***.

Paso 4: vista de la colección en Compass. El explorador de Compass muestra la base de datos `Universidad` y su colección `estudiantes`, con tres documentos y un tamaño de 4,10 kB (Figura 14).

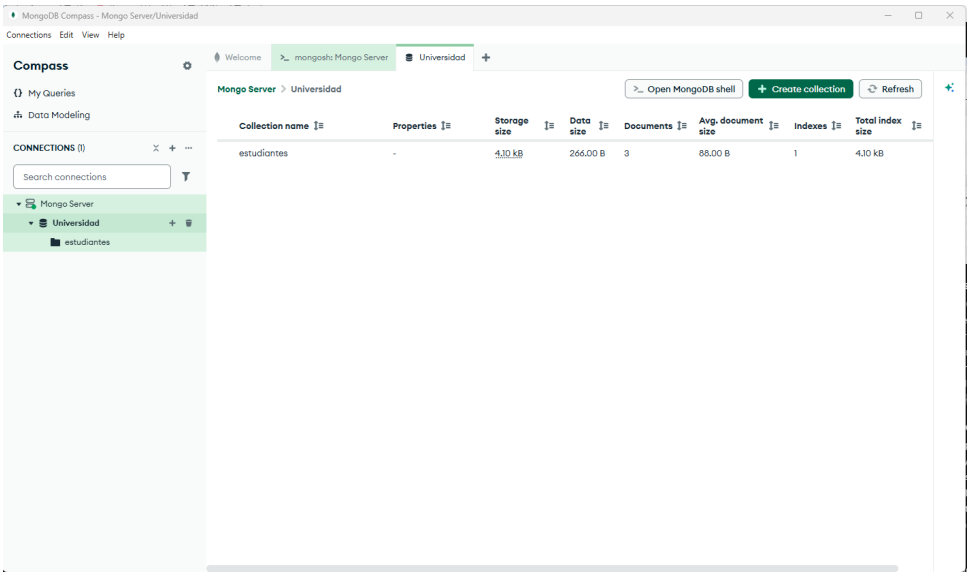


Figura 14. *Base* ***`Universidad`*** *y colección* ***`estudiantes`*** *en Compass.*

Paso 5: creación de la base y la colección. En `mongosh` se seleccionó la base con `use Universidad` y se creó la colección con `db.createCollection("estudiantes")`; el shell confirmó el cambio con `switched to db Universidad` (Figura 15).

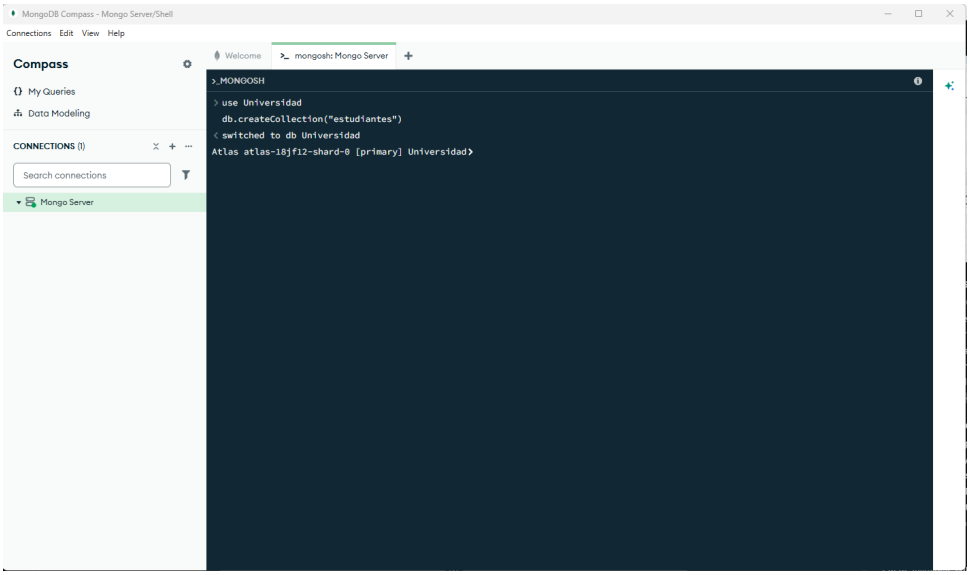


Figura 15. *Selección de la base y creación de la colección.*

Paso 6: vista de los documentos en Compass. La vista de documentos de Compass muestra los tres documentos insertados en la colección `estudiantes` (Figura 16).

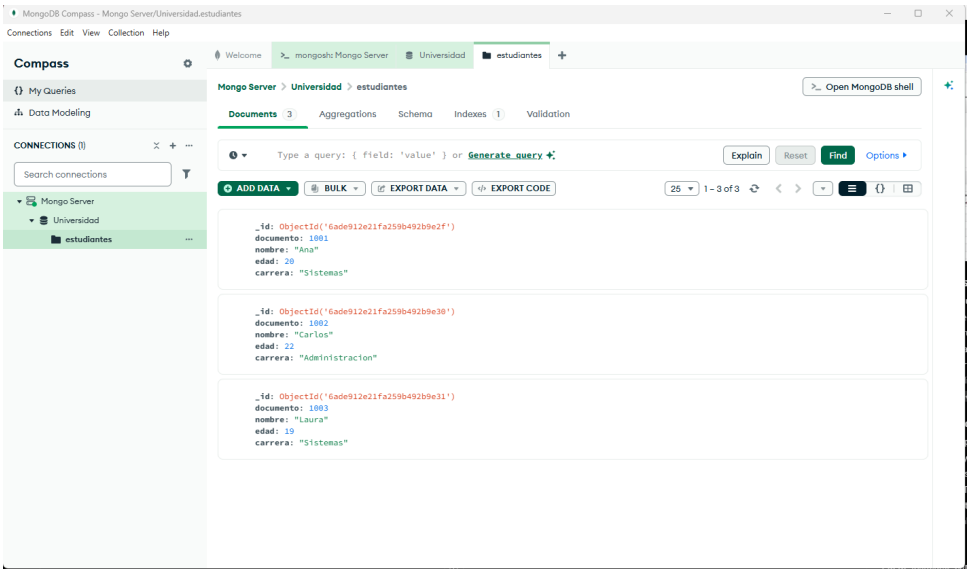


Figura 16. *Documentos de la colección* `estudiantes` *en Compass.*

Paso 7: consulta de todos los documentos. La consulta `db.estudiantes.find()` devolvió los tres documentos (Figura 17).

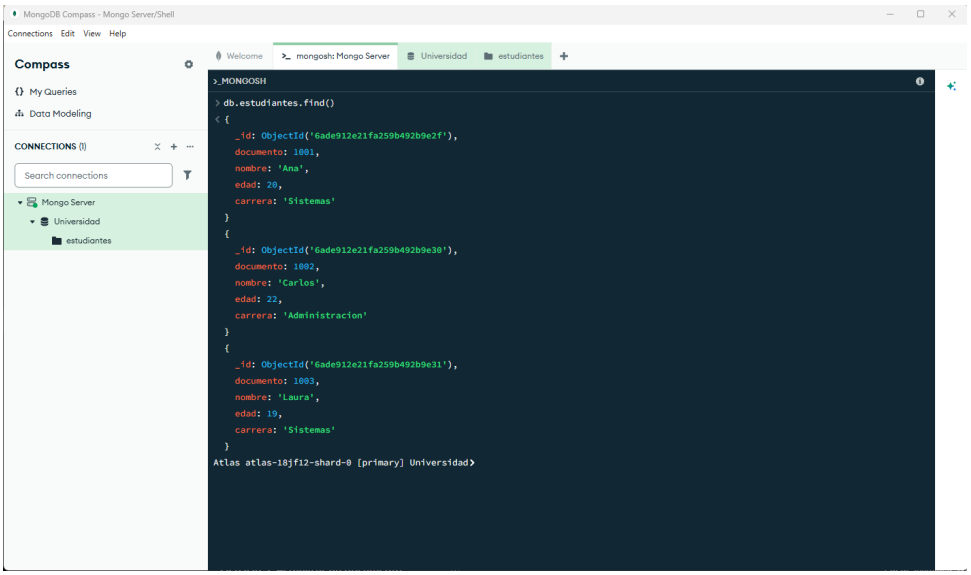


Figura 17. *Consulta* `db.estudiantes.find()`.

Paso 8: consulta filtrada. La consulta `db.estudiantes.find({ carrera: "Sistemas" })` devolvió dos documentos (Ana y Laura) (Figura 18).

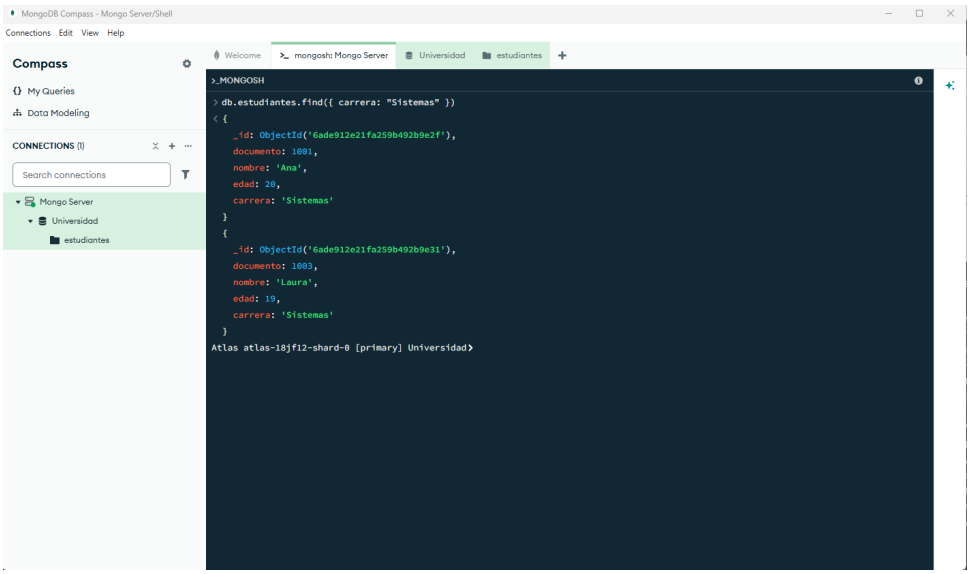


Figura 18. *Consulta filtrada por* `carrera: "Sistemas"`.

Paso 9: actualización de un documento. La operación `updateOne` sobre el documento 1001 cambió su edad a 21 (`matchedCount: 1`, `modifiedCount: 1`), y `findOne` confirmó el cambio (Figura 19).

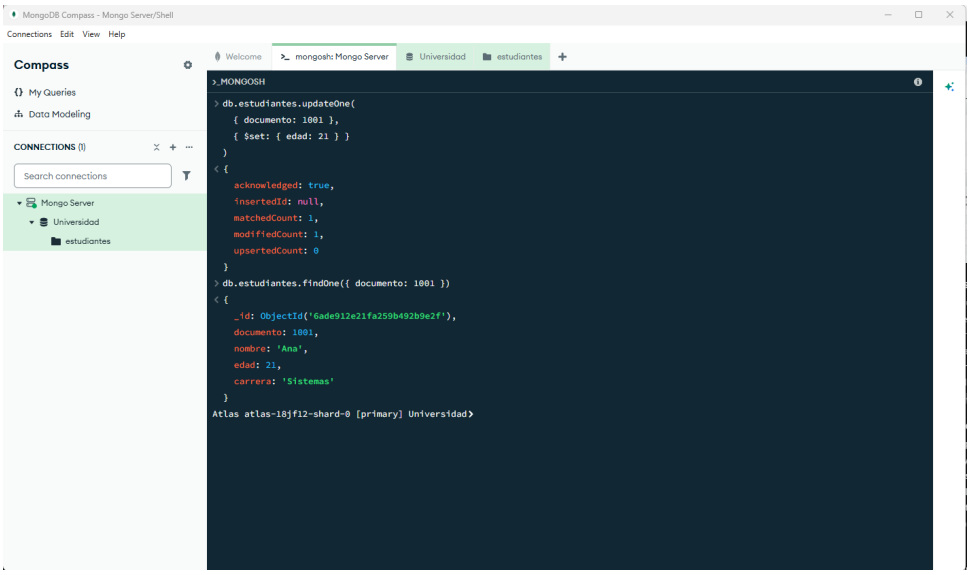


Figura 19. *Actualización con* `updateOne` *y verificación con* `findOne`*.*

Paso 10: eliminación de un documento. La operación `deleteOne` sobre el documento 1003 devolvió `deletedCount: 1`, y la consulta final `find()` mostró los dos documentos restantes (Figura 20).

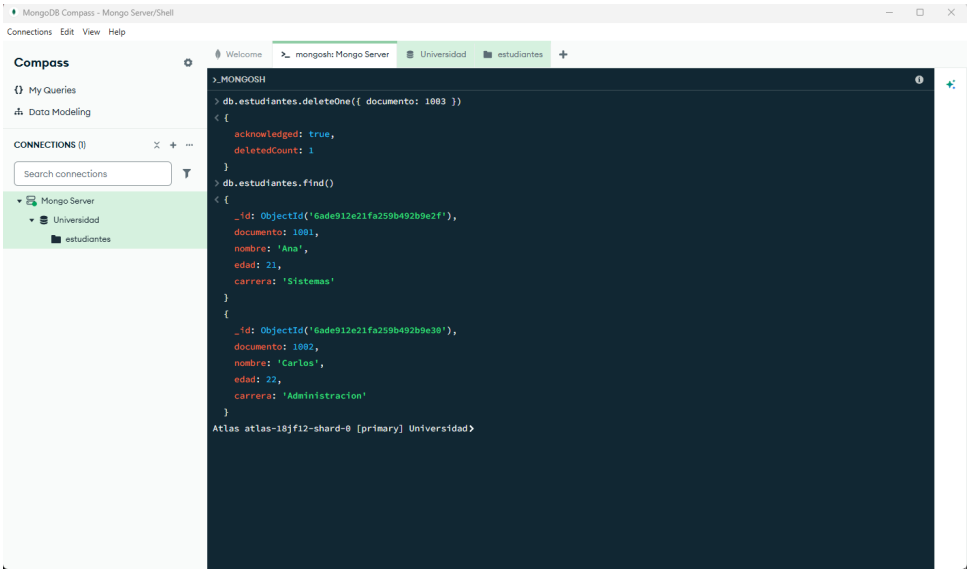


Figura 20. *Eliminación con* `deleteOne` *y consulta final* `find()`*.*

Paso 11: vista de los documentos restantes. La vista gráfica de Compass confirma que la colección conserva dos documentos tras la eliminación (Figura 21).

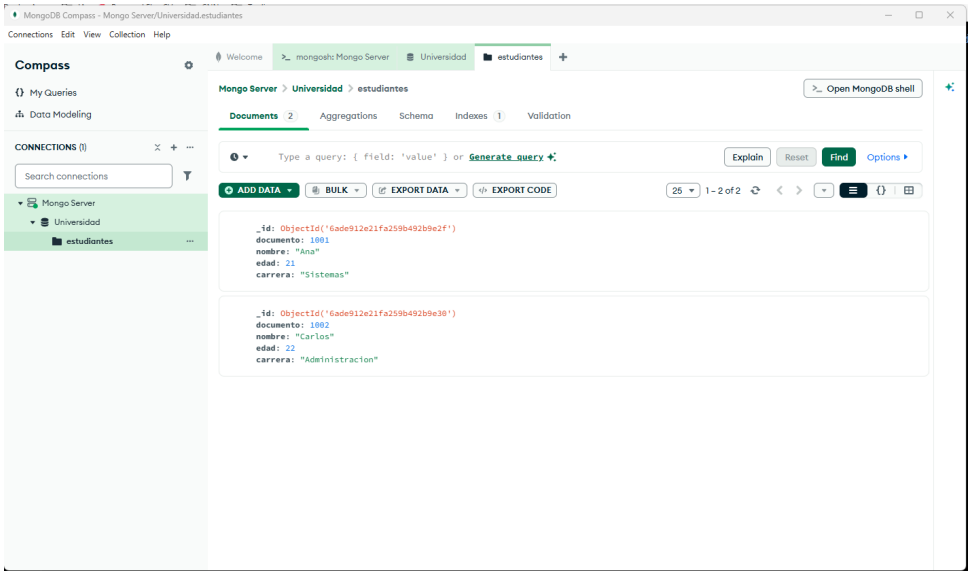


Figura 21. *Vista de Compass con los dos documentos restantes.*

Paso 12: estado del clúster. El panel del clúster `MongoTest` en MongoDB Atlas muestra el despliegue utilizado: versión 8.0.34, región us-east-1, nivel gratuito y tres nodos (Figura 22).

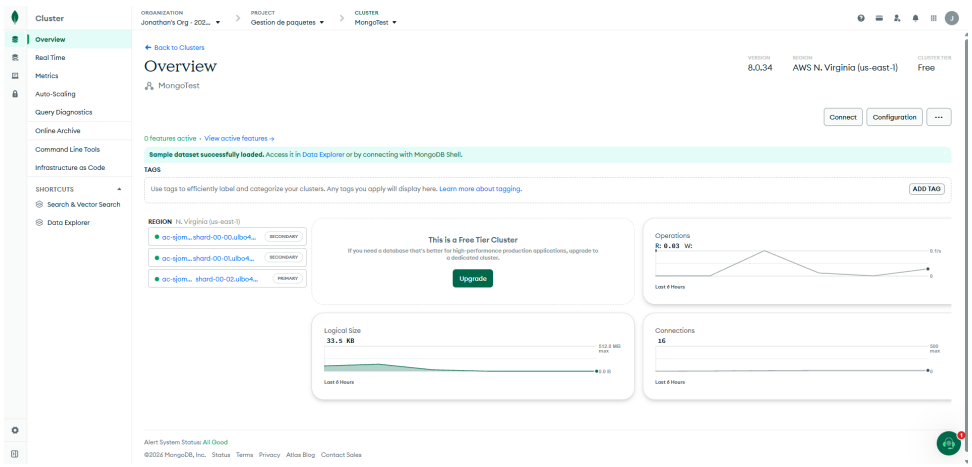


Figura 22. *Panel del clúster `MongoTest` en MongoDB Atlas.*

5 Resultados y análisis

En ambos motores se completó el ciclo CRUD sobre la entidad de estudiantes sin errores, partiendo de tres registros y finalizando con dos tras la eliminación de uno de ellos, y con la actualización del primer registro verificada en cada caso. La diferencia más evidente radica en el modelo de datos: CockroachDB exige definir previamente el esquema y la estructura fija de la tabla (`id`, `nombre`, `edad`, `carrera`, `semestre`), mientras que MongoDB permite insertar documentos directamente en la colección, sin una definición previa de campos (Carvalho et al., 2022).

En cuanto al lenguaje, las operaciones de CockroachDB se expresan en SQL declarativo (`CREATE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE`), lo que mantiene compatibilidad con

el ecosistema relacional (Reid, 2022). En MongoDB, en cambio, las operaciones se expresan mediante métodos del shell sobre la colección (`insertMany`, `find`, `updateOne`, `deleteOne`), con filtros expresados como objetos JSON. Ambas herramientas gráficas (DBeaver y Compass) ofrecen paneles de control que confirman el estado de los clústeres en la nube y las métricas de ejecución, lo que facilita la verificación de los resultados.

6 Conclusiones

La práctica permitió ejecutar y verificar el ciclo completo de gestión de datos en un motor NewSQL y en un motor NoSQL sobre un mismo caso de estudio. CockroachDB conservó el modelo relacional estricto y el lenguaje SQL, añadiendo la escalabilidad y la distribución propias de un sistema en la nube, mientras que MongoDB demostró la flexibilidad del modelo documental, en el que los registros se insertan y consultan sin una estructura rígida previa. Las herramientas de cliente (DBeaver, Compass y `mongosh`) y los paneles de los servicios en la nube permitieron confirmar de forma visual cada operación, y las capturas de pantalla recopiladas constituyen la evidencia de los procedimientos realizados. En conjunto, se evidencia que ambos paradigmas resuelven el mismo problema de almacenamiento desde enfoques complementarios, y que su elección depende del grado de estructura, de las garantías de consistencia y de la flexibilidad que requiera cada aplicación (Tankoano, 2025).

References

Carvalho, I., Sá, F., & Bernardino, J. (2022). NoSQL document databases assessment: Couchbase, couchdb, and MongoDB. In *Proceedings of the 11th international conference on data science, technology and applications* (pp. 557–564). Lisboa, Portugal: SCITEPRESS. doi: 10.5220/0011352700003269

Cockroach Labs. (2025). *Cockroachdb documentation.* Retrieved from `https://www.cockroachlabs.com/docs/` (Documentación en línea)

MongoDB, Inc. (2025). *Mongodb manual.* Retrieved from `https://www.mongodb.com/docs/manual/` (Documentación en línea)

Reid, R. (2022). *Practical cockroachdb.* New York, NY: Apress. doi: 10.1007/978-1-4842-8224-3

Tankoano, J. (2025). Multi-model database design and query processing in NewSQL DBMSs. *American Journal of Computer Science and Technology*, *8*(2), 52–71. doi: 10.11648/j.ajcst.20250802.12